

Order Imbalance

Raj Kundi

2026-08-26

Table of contents

1	Introduction	1
2	Intraday Features	2
2.1	Autocorrelation of Order Imbalance	11
2.2	Imbalance and Future Price Changes	12
2.2.1	Markov chain models	15
2.2.2	Maximum likelihood estimator	15
2.2.3	Estimation	15
2.2.4	Discrete to continuous time Markov models	17
2.2.5	Jointly modelling market orders	18
2.2.6	Modelling price jumps	29
3	Daily features	36
4	Optimal liquidation	36
4.1	Optimisation problem	36
4.1.1	Simulations	36

1 Introduction

Agents in optimal execution and market making make decisions based on three things: the

- midprice;
- arrival rates, and
- the agent's own inventory.

In some cases, these state variables can be supplemented by observables such as order flow, short term alpha and cointegration of prices.

We want to investigate the role the quoted volume order imbalance (or simply **order imbalance**) plays in optimal execution models. This is a measure of buy versus sell pressure in the book and *it contains predictive power on both the arrival rates of market orders and the direction and size of future price movements.*

We break down the analysis into the following:

1. how does the data evolve intraday;
2. models for order imbalance, arrival rates for market orders and prices;
3. daily features exhibited by order imbalance, and
4. implementation into optimal liquidation problems using limit orders.

2 Intraday Features

The limit order imbalance

$$\rho_t = \frac{V_t^b - V_t^a}{V_t^b + V_t^a},$$

which is just the ratio of the quoted volume imbalance to the total quoted volume, where V_t^b V_t^a denotes the value of limit orders posted on the bid and respectively ask sides of the book.

The volumes used could be:

- *at the touch*;
- the best n -levels of the book, or
- the volume posted within n ticks of the midprice.

```
trades <- fread(
  "data/SOLUSD_PERP-trades-2024-08.csv"
)

bookTicker <- fread(
  "data/SOLUSD_PERP-bookTicker-2024-08.csv",
  select = c(
    "best_bid_price", "best_ask_price",
    "best_bid_qty", "best_ask_qty", "transaction_time"
  )
)

# Half the quoted spread: what a marketable order concedes just by
# crossing.
```

```

bookTicker[, half_spread := (best_ask_price - best_bid_price) / 2]
bookTicker[, transaction_ts := as.POSIXct(
  transaction_time / 1000,
  origin = "1970-01-01",
  tz = "UTC"
)]

setorder(bookTicker, transaction_time)

# Calculations below use the full month of data. For scatter-style
# plots (one point per 100ms bin, ~26.7M points across a month) we
# instead plot only a single minute's worth of bins to keep figures
# small and legible.
plot_date <- as.POSIXct("2024-08-01", tz = "UTC")
plot_window_seconds <- 60
plot_date_range_ms <- as.numeric(c(
  plot_date, plot_date + plot_window_seconds
)) * 1000

# Calculate at the touch order imbalance:  $(V^b - V^a) / (V^b + V^a)$ .
bookTicker[
  ,
  imbalance := (best_bid_qty - best_ask_qty) / (best_bid_qty + best_ask_qty)
]

# Group by 100ms bins and calculate statistics
bookTicker[, time_bin := transaction_time %/% 100]

binned_data <- bookTicker[, .(
  avg_imbalance = mean(imbalance),
  first_timestamp = min(transaction_ts)
), by = time_bin]

# Create 5 equally spaced quantile regimes from -1 to 1
quantile_bounds <- seq(-1, 1, length.out = 6)
regime_labels <-
  c("Very Negative", "Negative", "Neutral", "Positive", "Very Positive")

binned_data[, regime := cut(
  avg_imbalance,
  breaks = quantile_bounds,
  labels = regime_labels,

```

```

    include.lowest = TRUE
  )]

print(paste("Total bins:", nrow(binned_data)))

```

```
[1] "Total bins: 14351665"
```

```

print(
  paste(
    "Imbalance range: [",
    round(min(binned_data$avg_imbalance), 4),
    ", ",
    round(max(binned_data$avg_imbalance), 4),
    "]",
    sep = ""
  )
)

```

```
[1] "Imbalance range: [-1, 1]"
```

```

# Create color palette for regimes
regime_colors <- c(
  "Very Negative" = "#d73027",
  "Negative" = "#fc8d59",
  "Neutral" = "#fee090",
  "Positive" = "#91bfdb",
  "Very Positive" = "#4575b4"
)

# One point per 100ms bin over a full month is far too many points to
# plot sensibly, so the scatter plots below use a single minute's
# bins.
plot_binned_data <- binned_data[
  first_timestamp >=
    plot_date & first_timestamp < plot_date + plot_window_seconds
]

# Plot 1: Time series with quantile regimes colored
p1 <- ggplot(plot_binned_data, aes(x = first_timestamp, y = avg_imbalance)) +
  geom_line(color = "black", linewidth = 0.2, alpha = 0.15) +
  geom_point(aes(color = regime), alpha = 0.6, size = 1.5) +

```

```

geom_hline(
  yintercept = 0, linetype = "dashed", color = "black",
  linewidth = 0.3, alpha = 0.5
) +
geom_hline(
  yintercept = quantile_bounds, linetype = "dotted", color = "gray",
  linewidth = 0.3, alpha = 0.3
) +
scale_color_manual(values = regime_colors) +
labs(
  title = "BookTicker Imbalance by 100ms Bins (Colored by Quantile Regime)",
  x = "Time",
  y = "Average Imbalance",
  color = "Regime"
) +
theme_minimal() +
theme(
  plot.title = element_text(size = 12, face = "bold"),
  axis.text.x = element_text(angle = 45, hjust = 1),
  panel.grid.major = element_line(color = "gray90"),
  panel.grid.minor = element_line(color = "gray95")
)

# Plot 2: Distribution histogram with regime boundaries
p2 <- ggplot(binned_data, aes(x = avg_imbalance)) +
  geom_histogram(
    bins = 50, alpha = 0.7, fill = "steelblue", color = "black",
    linewidth = 0.2
  ) +
  geom_vline(
    xintercept = quantile_bounds, linetype = "dashed", color = "red",
    linewidth = 0.8, alpha = 0.7
  ) +
  labs(
    title = "Distribution of Average Imbalance with Quantile Boundaries",
    x = "Average Imbalance",
    y = "Frequency"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(size = 12, face = "bold"),
    panel.grid.major = element_line(color = "gray90"),

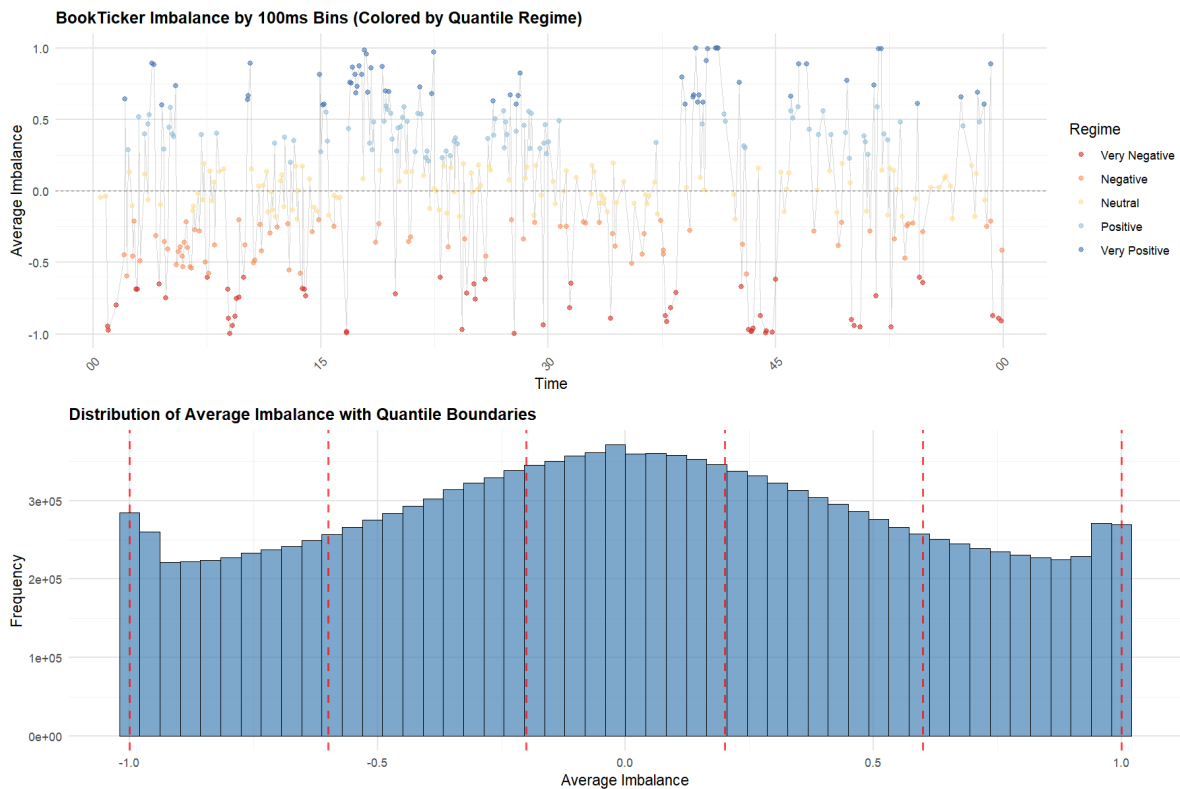
```

```

    panel.grid.minor = element_line(color = "gray95")
  )

gridExtra::grid.arrange(p1, p2, nrow = 2)

```



```

# Bin trades into 100ms intervals and count by side
trades[, time_bin := time %/% 100]

trades_binned <- trades[, .(
  n_trades = .N,
  n_buys = sum(is_buyer_maker == FALSE),
  n_sells = sum(is_buyer_maker == TRUE)
), by = time_bin]

# Merge trades with imbalance bins
binned_data <- merge(binned_data, trades_binned, by = "time_bin", all.x = TRUE)

# Fill NAs with 0 (bins with no trades)
binned_data[is.na(n_trades), n_trades := 0]

```

```

binned_data[is.na(n_buys), n_buys := 0]
binned_data[is.na(n_sells), n_sells := 0]

print(paste("Bins with trades:", sum(binned_data$n_trades > 0)))

```

```
[1] "Bins with trades: 1968382"
```

```
print(paste("Total trades:", sum(binned_data$n_trades)))
```

```
[1] "Total trades: 7919549"
```

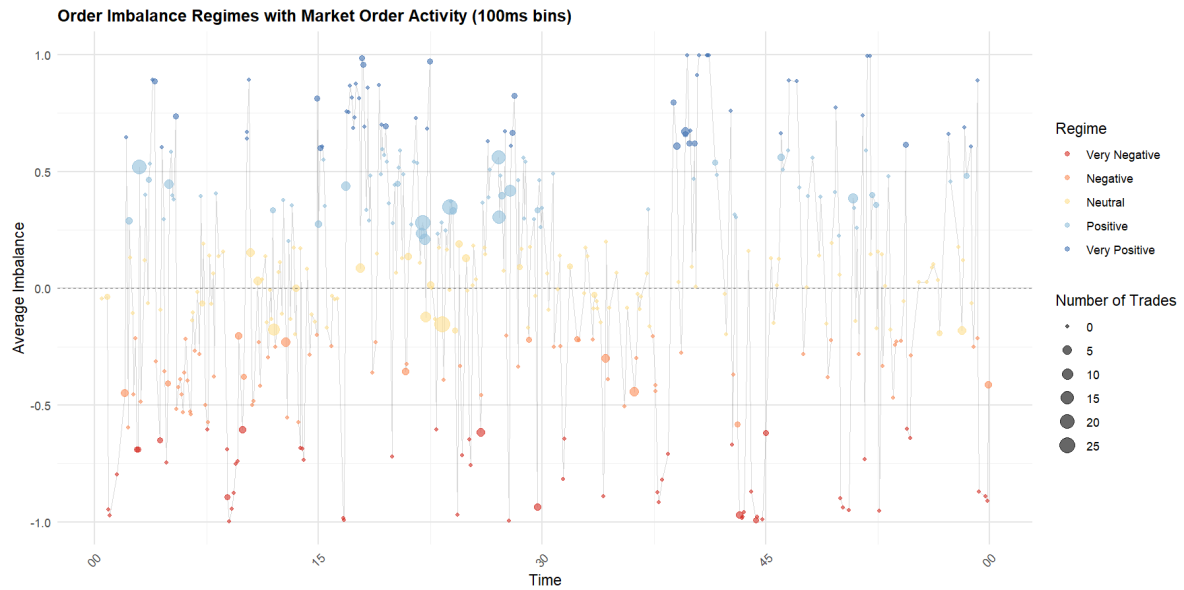
```

# Recompute the single-minute subset now that trade counts have been
# merged in.
plot_binned_data <- binned_data[
  first_timestamp >=
    plot_date & first_timestamp < plot_date + plot_window_seconds
]

# Plot with trade activity overlaid
p3 <- ggplot(plot_binned_data, aes(x = first_timestamp, y = avg_imbalance)) +
  geom_line(color = "black", linewidth = 0.2, alpha = 0.15) +
  geom_point(aes(color = regime, size = n_trades), alpha = 0.6) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "black",
    linewidth = 0.3, alpha = 0.5)
) +
  scale_color_manual(values = regime_colors) +
  scale_size_continuous(name = "Number of Trades", range = c(1, 5)) +
  labs(
    title = "Order Imbalance Regimes with Market Order Activity (100ms bins)",
    x = "Time",
    y = "Average Imbalance",
    color = "Regime",
    size = "Trades"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(size = 12, face = "bold"),
    axis.text.x = element_text(angle = 45, hjust = 1),
    panel.grid.major = element_line(color = "gray90"),
    panel.grid.minor = element_line(color = "gray95")
  )

```

```
)  
  
print(p3)
```



```
# Summary of trades by regime
trades_by_regime <- binned_data[, .(
  n_bins = .N,
  total_trades = sum(n_trades),
  total_buys = sum(n_buys),
  total_sells = sum(n_sells),
  avg_trades_per_bin = mean(n_trades),
  avg_imbalance = mean(avg_imbalance)
), by = regime]

knitr::kable(
  trades_by_regime,
  caption = "Trade Activity by Order Imbalance Regime"
)
```

Table 1: Trade Activity by Order Imbalance Regime

regime	n_bins	to- tal_trades	to- tal_buys	total_sells	avg_trades_per_bin	avg_imbal- ance
Neutral	3490959	2687536	1303446	1384090	0.7698561	0.0002898
Very Negative	2478364	859900	150837	709063	0.3469628	-0.8116603
Negative	2937750	1800591	578288	1222303	0.6129150	-0.3898066
Very Positive	2497285	847249	680840	166409	0.3392680	0.8113238
Positive	2947307	1724273	1134635	589638	0.5850334	0.3903157

```
# Prepare data for stacked bar chart
trades_regime_long <- trades_by_regime[, .(regime, total_buys, total_sells)]
trades_regime_long <- melt(
  trades_regime_long,
  id.vars = "regime",
  variable.name = "side",
  value.name = "count"
)

trades_regime_long[
  ,
  side := factor(
    side, levels = c("total_buys", "total_sells"),
    labels = c("Market Buys", "Market Sells")
  )
]

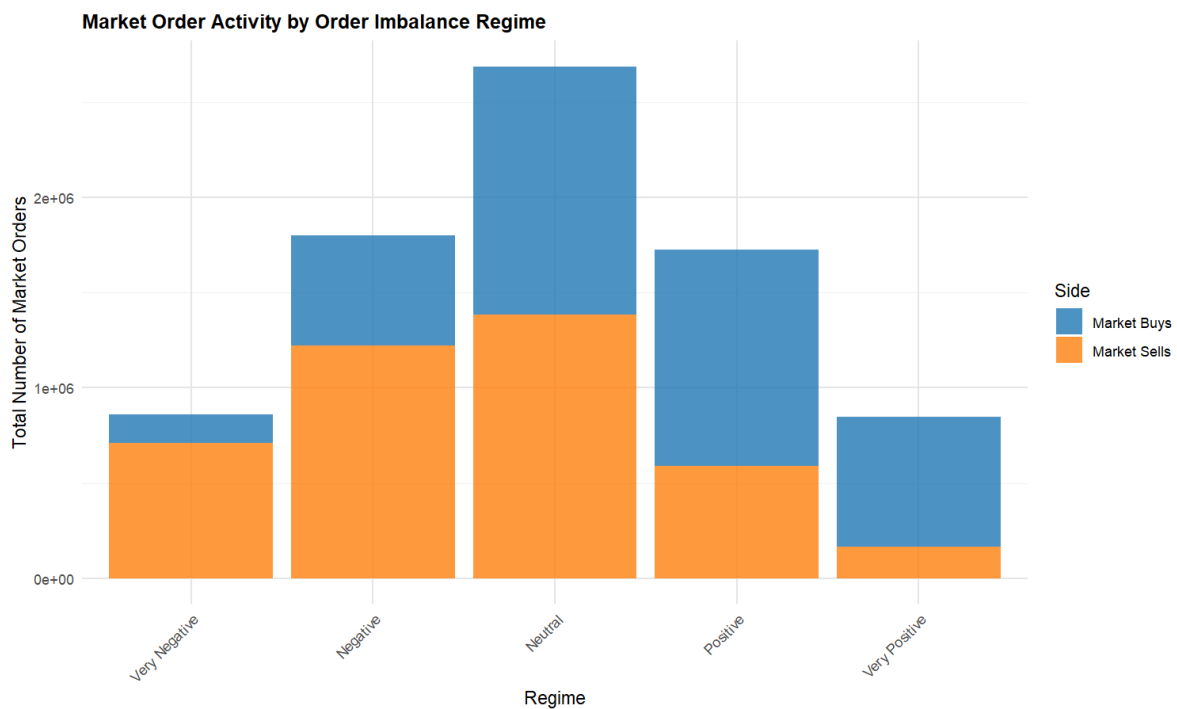
# Stacked bar chart
p4 <- ggplot(
  trades_regime_long, aes(x = factor(regime, levels = regime_labels),
    y = count, fill = side)
) +
  geom_col(position = "stack", alpha = 0.8) +
  scale_fill_manual(
    values = c("Market Buys" = "#1f77b4", "Market Sells" = "#ff7f0e")
  ) +
  labs(
    title = "Market Order Activity by Order Imbalance Regime",
    x = "Regime",
    y = "Total Number of Market Orders",
```

```

    fill = "Side"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(size = 12, face = "bold"),
    axis.text.x = element_text(angle = 45, hjust = 1),
    panel.grid.major = element_line(color = "gray90"),
    panel.grid.minor = element_line(color = "gray95")
  )

print(p4)

```



```

# Summary statistics by regime
summary_stats <- binned_data[, .(
  Count = .N,
  Mean = mean(avg_imbalance),
  Std = sd(avg_imbalance),
  Min = min(avg_imbalance),
  Max = max(avg_imbalance)
), by = regime]

```

```
knitr::kable(summary_stats, caption = "Summary Statistics by Quantile Regime")
```

Table 2: Summary Statistics by Quantile Regime

regime	Count	Mean	Std	Min	Max
Neutral	3490959	0.0002898	0.1145010	-0.1999999	0.2000000
Very Negative	2478364	-0.8116603	0.1235807	-0.9999731	-0.6000000
Negative	2937750	-0.3898066	0.1149620	-0.5999999	-0.2000000
Very Positive	2497285	0.8113238	0.1229248	0.6000004	0.9999753
Positive	2947307	0.3903157	0.1148834	0.2000002	0.6000000

2.1 Autocorrelation of Order Imbalance

We examine the persistence of order imbalance by computing its autocorrelation function (ACF) over 2,000 lags of the 100ms bins, i.e. roughly the first 200 seconds.

```
setorder(binned_data, time_bin)

acf_result <- acf(
  binned_data$avg_imbalance,
  lag.max = 2000,
  plot = FALSE,
  na.action = na.pass
)

acf_df <- data.table(
  lag = as.integer(acf_result$lag[, 1, 1]),
  acf = as.numeric(acf_result$acf[, 1, 1])
)
acf_df[, lag_seconds := lag * 0.1]

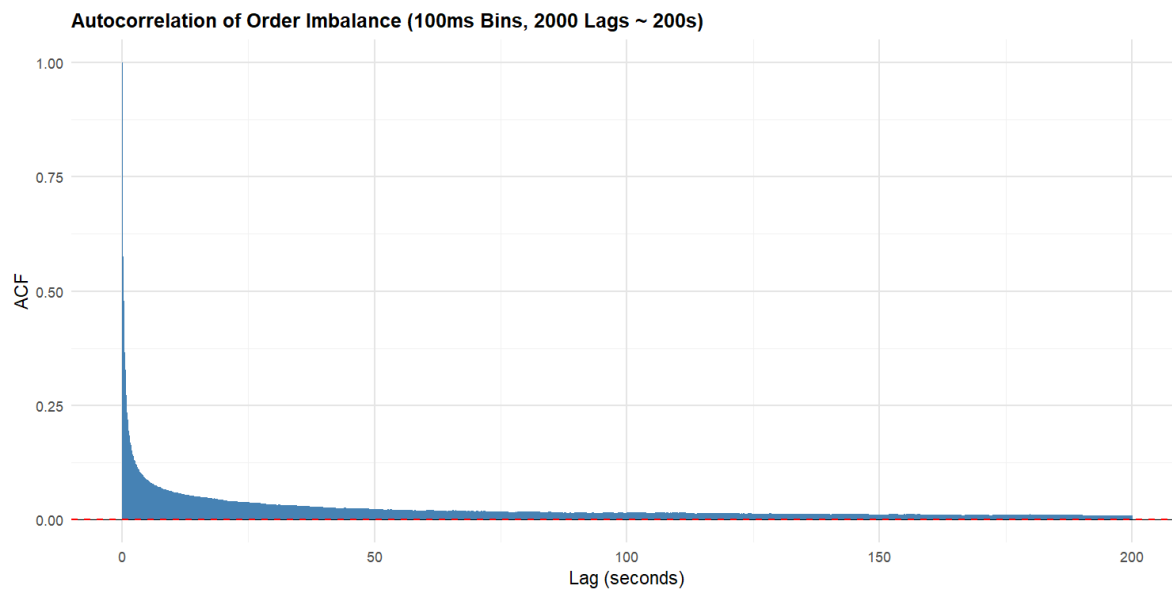
# 95% confidence bound under the white-noise null
ci_bound <- qnorm(0.975) / sqrt(acf_result$n.used)

p5 <- ggplot(acf_df, aes(x = lag_seconds, y = acf)) +
  geom_segment(
    aes(xend = lag_seconds, yend = 0), color = "steelblue", linewidth = 0.3
  ) +
  geom_hline(yintercept = 0, color = "black", linewidth = 0.3) +
  geom_hline(
```

```

    yintercept = c(-ci_bound, ci_bound), linetype = "dashed", color = "red",
    linewidth = 0.5
  ) +
  labs(
    title = "Autocorrelation of Order Imbalance (100ms Bins, 2000 Lags ~ 200s)",
    x = "Lag (seconds)",
    y = "ACF"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(size = 12, face = "bold"),
    panel.grid.major = element_line(color = "gray90"),
    panel.grid.minor = element_line(color = "gray95")
  )
print(p5)

```



2.2 Imbalance and Future Price Changes

We measure the correlation between the current order imbalance and the subsequent midprice change, as a function of the forecast horizon n (in 100ms intervals). Since the midprice is frequently unchanged between consecutive bins at this frequency, we condition on the price having actually moved by horizon n .

```

# End-of-bin midprice for each 100ms bin
bookTicker[, midprice := (best_bid_price + best_ask_price) / 2]

midprice_binned <- bookTicker[, .(
  midprice = data.table::last(midprice)
), by = time_bin]

binned_data <- merge(
  binned_data, midprice_binned, by = "time_bin", all.x = TRUE
)
setorder(binned_data, time_bin)

n_values <- 1:50 # up to 50 intervals ahead (5 seconds)

cor_results <- rbindlist(lapply(n_values, function(n) {
  price_change <-
    shift(binned_data$midprice, n = n, type = "lead") - binned_data$midprice
  imbalance <- binned_data$avg_imbalance

  # Condition on a price change actually occurring over the horizon
  valid <- !is.na(price_change) & !is.na(imbalance) & price_change != 0

  data.table(
    n = n,
    n_seconds = n * 0.1,
    correlation = cor(imbalance[valid], price_change[valid]),
    n_obs = sum(valid)
  )
}))

knitr::kable(
  head(cor_results, 10),
  caption = "Imbalance vs. Future Price Change Correlation (First 10 Horizons)",
  digits = 4
)

```

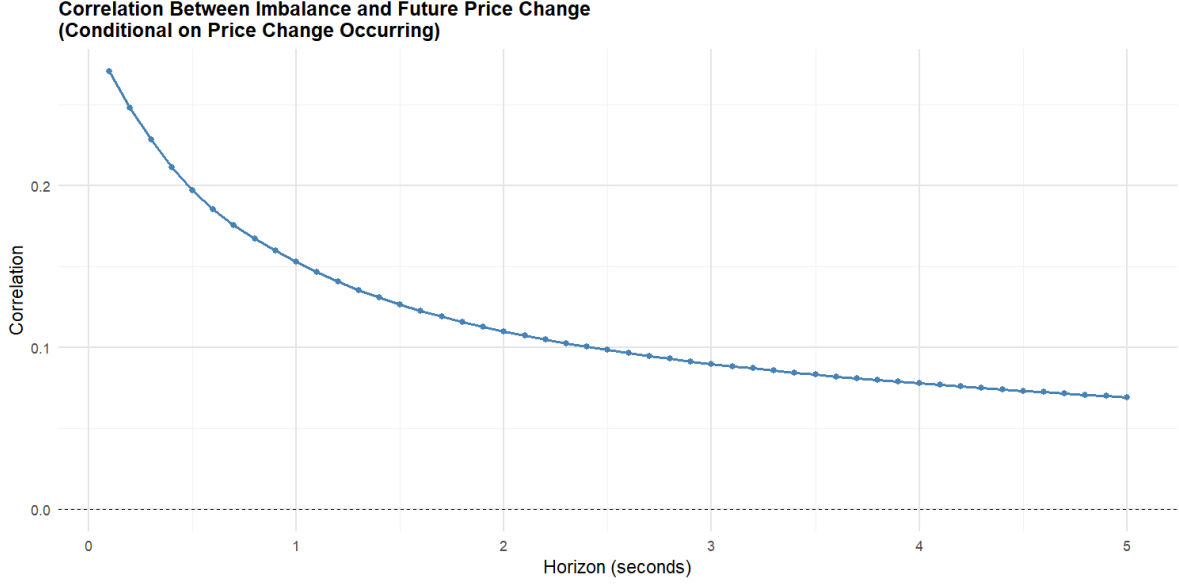
Table 3: Imbalance vs. Future Price Change Correlation (First 10 Horizons)

n	n_seconds	correlation	n_obs
1	0.1	0.2705	5390458
2	0.2	0.2478	7038387

n	n_seconds	correlation	n_obs
3	0.3	0.2281	8094802
4	0.4	0.2110	8868697
5	0.5	0.1969	9481977
6	0.6	0.1852	9986391
7	0.7	0.1754	10411705
8	0.8	0.1670	10776034
9	0.9	0.1596	11090706
10	1.0	0.1528	11364850

```
p6 <- ggplot(cor_results, aes(x = n_seconds, y = correlation)) +
  geom_line(color = "steelblue", linewidth = 0.8) +
  geom_point(color = "steelblue", size = 1.5) +
  geom_hline(
    yintercept = 0, linetype = "dashed", color = "black", linewidth = 0.3
  ) +
  labs(
    title = paste0(
      "Correlation Between Imbalance and Future Price",
      " Change\n(Conditional on Price Change Occurring)"
    ),
    x = "Horizon (seconds)",
    y = "Correlation"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(size = 12, face = "bold"),
    panel.grid.major = element_line(color = "gray90"),
    panel.grid.minor = element_line(color = "gray95")
  )

print(p6)
```



2.2.1 Markov chain models

We build a simple markov chain model for order imbalance and show how to calibrate it to market data. Let $Z_t \in \{1, \dots, K\}$ denote the order imbalance regime observed at time $t \in \{1, 2, \dots, T\}$. We assume that the order imbalance is described by a Markov chain with transition matrix A .

2.2.2 Maximum likelihood estimator

Let z_t denote a sequence of observations from the Markov chain Z . The maximum likelihood estimator for A is known to be

$$\hat{A} = \frac{n_{i,j}}{\sum_{j=1}^K n_{i,j}},$$

where $n_{i,j} = \sum_{t=2}^T \chi_{Z_{t-1}=i, Z_t=j}$, the number of observed transitions from regime i to j .

2.2.3 Estimation

Using orderbook snapshots (volume at the touch) averaged over 100ms bins, and then smoothed with a 10s trailing moving average to damp the tick-by-tick flicker at the touch. We then bin the order imbalance into 5 equally spaced regimes, and find the following estimator for the transition matrix, where consecutive observations are $\Delta T = 0.1s$ apart:

```

setorder(binned_data, time_bin)

# 100-bin (10s) trailing moving average of the 100ms-binned imbalance.
binned_data[, smoothed_imbalance := zoo::rollmean(
  avg_imbalance, k = 100, fill = NA, align = "right"
)]

binned_data[, smoothed_regime := cut(
  smoothed_imbalance,
  breaks = quantile_bounds,
  labels = regime_labels,
  include.lowest = TRUE
)]

regime_seq <- na.omit(binned_data$smoothed_regime)

transition_counts <- table(
  from = head(regime_seq, -1),
  to = tail(regime_seq, -1)
)

# MLE: row-normalize observed transition counts,  $A_{\hat{}}[i, j] = n_{ij} /$ 
#  $\sum_j n_{ij}$ .
transition_matrix <- sweep(
  unclass(transition_counts), 1, rowSums(transition_counts), FUN = "/"
)

knitr::kable(
  transition_matrix,
  caption =
    "Maximum Likelihood Estimate of the Order Imbalance Transition Matrix",
  digits = 4
)

```

Table 4: Maximum Likelihood Estimate of the Order Imbalance Transition Matrix

	Very Negative	Negative	Neutral	Positive	Very Positive
Very Negative	0.9698	0.0302	0.0000	0.0000	0.0000
Negative	0.0011	0.9790	0.0199	0.0000	0.0000
Neutral	0.0000	0.0055	0.9890	0.0055	0.0000
Positive	0.0000	0.0000	0.0194	0.9795	0.0011

	Very Negative	Negative	Neutral	Positive	Very Positive
Very Positive	0.0000	0.0000	0.0000	0.0283	0.9717

2.2.4 Discrete to continuous time Markov models

To use the tools of continuous time stochastic control theory, we need to move from discrete to continuous models. There exists transformations from discrete to continuous time models and estimations. For a continuous time Markov model $\mathcal{Z} = (Z_t)$, with $Z_t \in \{1, \dots, K\}$, it has a generator matrix B which produces transition probabilities

$$\mathbb{P}(Z_t = j | Z_s = i) = [\exp(B(t - s))]_{i,j}, \quad t \geq s.$$

The continuous time model can be estimated by the discrete time one by matching the estimated transition probability in the discrete setting from Section 2.2.3. Specifically, we set

$$\exp(\hat{B}\Delta T) = \hat{A}, \quad \hat{B} = \frac{1}{\Delta T} \log(\hat{A})$$

```
# The discrete transition matrix A_hat (@sec-estimation) links
# consecutive 100ms bins, so Delta T is 0.1 seconds.
delta_t <- 0.1

generator_matrix <- expm::logm(transition_matrix) / delta_t

knitr::kable(
  generator_matrix,
  caption = paste0(
    "Estimated Generator Matrix  $\hat{B}$  of",
    " the Continuous Time Order Imbalance Model"
  ),
  digits = 4
)
```

Table 5: Estimated Generator Matrix $\hat{B}$ of the Continuous Time Order Imbalance Model

-0.3064	0.3095	-0.0031	0.0000	0.0000
0.0116	-0.2135	0.2025	-0.0006	0.0000
0.0000	0.0561	-0.1115	0.0555	0.0000
0.0000	-0.0006	0.1969	-0.2079	0.0116
0.0000	0.0000	-0.0029	0.2904	-0.2876

Each row of a valid generator matrix should sum to zero. As a sanity check on the matrix logarithm:

```
print(paste(
  "Max absolute row sum (should be ~0 for a valid generator):",
  round(max(abs(rowSums(generator_matrix))), 6)
))
```

```
[1] "Max absolute row sum (should be ~0 for a valid generator): 0"
```

The diagonal elements represent minus one times the sum of the rate of transition out of the corresponding regime, so we can see that the chain tends to flow out of ask-heavy regimes towards the slightly bid heavy regimes. Once there it tends to flow back into the neutral regime at a slower rate.

2.2.5 Jointly modelling market orders

We extend the continuous-time Markov model of the previous section to include the modelling of MO arrivals. We also note that the continuous-time approximation above has one important flaw – the discrete-time model is estimated from average order imbalance over a 100ms window, therefore a continuous-time limit based on this estimate will not account properly for the interdependence of overlapping time windows.

To address these issues, we assume that, conditional on being within a given regime k , market buy and sell orders arrive independently at the arrival times of independent Poisson processes with rates λ_k^+ and λ_k^- , respectively. That is, the counting processes $M^\pm = \{M_t^\pm\}_{0 \leq t \leq T}$ of market buy (sell) orders are doubly stochastic Poisson processes with activity rates $\Lambda_t^\pm = \lambda_{Z_t}^\pm$, where $\lambda_1^\pm, \lambda_2^\pm, \dots, \lambda_K^\pm$ denote the activity rates in the various regimes. In addition, the order imbalance will be observed at every event, and the inter-arrival times of the events will play a role.

The regime path is therefore reconstructed from the *instantaneous* imbalance at the touch, updated at every book event, rather than from a time-averaged imbalance. Since the feed is stamped to the millisecond, the path can only be resolved to 1ms, so we keep the last update within each millisecond and take the regime to be constant in between.

```
# Regime at every book update, using the instantaneous imbalance at
# the touch.
bookTicker[, event_regime := cut(
  imbalance,
  breaks = quantile_bounds,
  labels = regime_labels,
```

```

    include.lowest = TRUE
  )]

# The feed is stamped to the millisecond, so keep the last update in
# each millisecond: this is the finest resolution at which the path is
# identified.
regime_path <- bookTicker[
  transaction_time != shift(transaction_time, type = "lead", fill = -1),
  .(t = as.numeric(transaction_time), regime = event_regime)
]

# Epochs are the maximal runs of a constant regime. tau is the
# switching time that starts the epoch, tau_next the one that ends it.
regime_path[, epoch_id := rleid(regime)]
epochs <- regime_path[, .(regime = regime[1L], tau = t[1L]), by = epoch_id]

epochs[, tau_next := shift(tau, type = "lead")]
epochs[, to_regime := shift(regime, type = "lead")]
epochs <- epochs[!is.na(tau_next)] # drop the censored final epoch
epochs[, dur := (tau_next - tau) / 1000]

# Feed outages show up as isolated multi-minute epochs during which
# nothing is observed. They would inflate the time-in-regime
# denominators, so drop them together with the switch that terminates
# them.
outage_seconds <- 60
n_outage <- sum(epochs$dur > outage_seconds)
t_outage <- sum(epochs$dur[epochs$dur > outage_seconds])
epochs <- epochs[dur <= outage_seconds]

# Market orders. is_buyer_maker == FALSE means the buyer lifted the
# ask, i.e. a market buy. Each MO is attached to the epoch prevailing
# at its arrival.
mo_events <- data.table(
  t = as.numeric(trades$time),
  side = fifelse(trades$is_buyer_maker, "sell", "buy")
)
setorder(mo_events, t)

setkey(epochs, tau)
mo_events[, ep_idx := epochs[.(mo_events$t), roll = TRUE, which = TRUE]]
mo_events <- mo_events[!is.na(ep_idx)]

```

```

mo_events[, `:=`(
  regime = epochs$regime[ep_idx],
  tau_next = epochs$tau_next[ep_idx]
)]
mo_events <- mo_events[t < tau_next] # drops MOs inside discarded epochs

rm(regime_path)
invisible(gc())

```

```

fig_start <- as.numeric(as.POSIXct("2024-08-05 12:00:00", tz = "UTC")) * 1000
fig_seconds <- 2

epoch_window <- epochs[tau >= fig_start & tau < fig_start + fig_seconds * 1000]
mo_window <- mo_events[t >= fig_start & t < fig_start + fig_seconds * 1000]

epoch_window[
  `:=`(t_rel = (tau - fig_start) / 1000, level = as.integer(regime))
]
mo_window[, t_rel := (t - fig_start) / 1000]

p_epochs <- ggplot(epoch_window, aes(x = t_rel, y = level)) +
  geom_vline(
    xintercept = epoch_window$t_rel, color = "gray85", linewidth = 0.2
  ) +
  geom_step(direction = "hv", color = "gray30", linewidth = 0.4) +
  geom_point(aes(color = regime), size = 1.8) +
  geom_point(
    data = mo_window,
    aes(x = t_rel, y = 0.45, fill = side, shape = side),
    inherit.aes = FALSE, size = 2.2, color = "black", stroke = 0.3
  ) +
  scale_color_manual(values = regime_colors, guide = "none") +
  scale_fill_manual(
    values = c(buy = "#1f77b4", sell = "#ff7f0e"),
    labels = c(buy = "Market Buy", sell = "Market Sell"), name = "Arrivals"
  ) +
  scale_shape_manual(
    values = c(buy = 24, sell = 25),
    labels = c(buy = "Market Buy", sell = "Market Sell"), name = "Arrivals"
  ) +
  scale_y_continuous(

```

```

    breaks = 1:5, labels = regime_labels, limits = c(0.3, 5.3)
  ) +
  labs(
    title = "Regime Path, Switching Times and Market Order Arrivals",
    x = "Seconds from 2024-08-05 12:00:00 UTC",
    y = NULL
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(size = 12, face = "bold"),
    panel.grid.minor = element_blank()
  )
print(p_epochs)

```

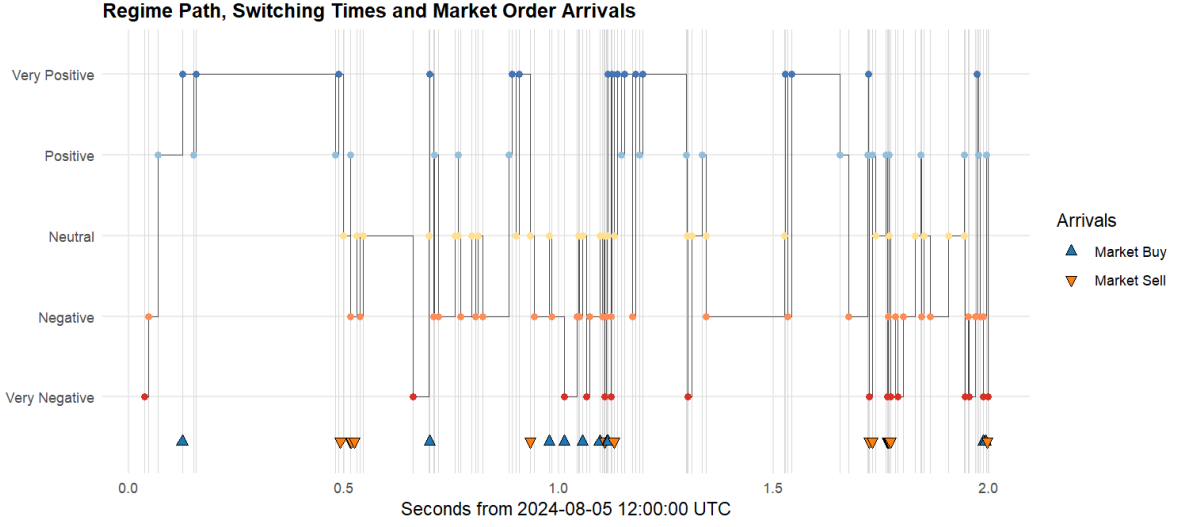


Figure 1: A two-second window of the regime path. Each step is an epoch $[\tau_r, \tau_{r+1})$, delimited by the grey switching times τ_r ; triangles mark market buy (up) and market sell (down) arrivals, which are Poisson with a regime-dependent rate within each epoch.

Referring to Figure 1, let $\tau_1, \tau_2, \dots, \tau_N$ denote the switching times of the regimes, i.e. the times at which the continuous-time Markov chain Z changes (with generator matrix denoted by B). We call the time interval $[\tau_r, \tau_{r+1})$ the r^{th} epoch ($r = 0, \dots, N$) with $\tau_{N+1} = T$ the end of the observation time horizon. Next, let $\{b_{r,1}, b_{r,2}, \dots, b_{r,m_r}\}$ and $\{s_{r,1}, s_{r,2}, \dots, s_{r,n_r}\}$ denote the arrival times of buy and sell MOs, respectively, during the r^{th} epoch. Then, since arrivals within an epoch of buys and sells are i.i.d., the likelihood within the r^{th} -epoch is

$$\begin{aligned}
L_r &= \underbrace{\lambda_{Z_{\tau_r}}^+ e^{-\lambda_{Z_{\tau_r}}^+ (b_{r,1} - b_{r,0})} \times \dots \times \lambda_{Z_{\tau_r}}^+ e^{-\lambda_{Z_{\tau_r}}^+ (b_{r,m_r} - b_{r,m_r-1})}}_{\text{buy arrivals}} \\
&\quad \times \underbrace{e^{-\lambda_{Z_{\tau_r}}^+ (\tau_{r+1} - b_{r,m_r})}}_{\text{no more buys}} \\
&\quad \times \underbrace{\lambda_{Z_{\tau_r}}^- e^{-\lambda_{Z_{\tau_r}}^- (s_{r,1} - s_{r,0})} \times \dots \times \lambda_{Z_{\tau_r}}^- e^{-\lambda_{Z_{\tau_r}}^- (s_{r,n_r} - s_{r,n_r-1})}}_{\text{sell arrivals}} \\
&\quad \times \underbrace{e^{-\lambda_{Z_{\tau_r}}^- (\tau_{r+1} - s_{r,n_r})}}_{\text{no more sells}} \\
&= \left(\lambda_{Z_{\tau_r}}^+ \right)^{M_r^+} e^{-\lambda_{Z_{\tau_r}}^+ (\tau_{r+1} - \tau_r)} \times \left(\lambda_{Z_{\tau_r}}^- \right)^{M_r^-} e^{-\lambda_{Z_{\tau_r}}^- (\tau_{r+1} - \tau_r)}.
\end{aligned}$$

Here, we set $b_{r,0} = s_{r,0} = \tau_r$ for notational convenience. In the first equality, the first (second) line represents the likelihood of the sequence of buy (sell) orders. In each line, the first to second last terms, of the form $\lambda_{Z_{\tau_r}}^\pm e^{-\lambda_{Z_{\tau_r}}^\pm (t_{r,l} - t_{r,l-1})}$, represent the survival since the last order arrival, and then the arrival of an MO at time $t_{r,l}$. The last term in each line represents the probability that no event arrives between the last buy (sell) MO and the time at which the Markov chain switches. The second equality is obtained by re-arranging the terms and letting $M_r^\pm$ denote the number of buy and sell MOs which arrive in the r^{th} -epoch.

The full likelihood is obtained by sewing together the within epoch likelihoods with the transition probability and the arrival of orders that occur after the last regime change, but before the sample ends. In all,

$$\begin{aligned}
L &= L_0 \times [e^{B\tau_1}]_{Z_{\tau_0} Z_{\tau_1}} \times [B]_{Z_{\tau_0} Z_{\tau_1}} \times \\
&\quad \dots \times L_{N-1} \times [e^{B(\tau_N - \tau_{N-1})}]_{Z_{\tau_{N-1}} Z_{\tau_N}} \times [B]_{Z_{\tau_{N-1}} Z_{\tau_N}}.
\end{aligned}$$

Recall that $\tau_0 = 0$ and $\tau_{N+1} = T$ is the time of the last MO (buy or sell) after the last regime change. We in fact exclude the data from this last regime change to avoid some issues with censored data – which render the maximisation analytically intractable, although it is still amenable to numerical methods. Next, recall that a continuous-time Markov chain generator matrix has rows which sum to zero, and the non-diagonal elements are non-negative. Hence, we write the generator matrix B as

$$B = \begin{pmatrix} -\Lambda_1 & \lambda_{12} & \lambda_{13} & \dots & \lambda_{1K} \\ \lambda_{21} & -\Lambda_2 & \lambda_{23} & \dots & \lambda_{2K} \\ \lambda_{31} & \lambda_{32} & -\Lambda_3 & \dots & \lambda_{3K} \\ \vdots & \vdots & & \ddots & \vdots \\ \lambda_{K1} & \lambda_{K2} & \lambda_{K3} & \dots & -\Lambda_K \end{pmatrix},$$

where $\Lambda_i := \sum_{j \neq i}^K \lambda_{ij}$ represents the total rate of outflow from regime i . The survival probability of the time τ_i at which the chain transitions out of regime i is given by

$$\mathbb{P}(\tau_i > t \mid Z_s = i) = e^{-\Lambda_i(t-s)}, \quad s \leq t,$$

and conditional on a transition occurring, the chain will switch from regime i to regime j with probability

$$[P]_{ij} = \frac{\lambda_{ij}}{\Lambda_i}, \quad \text{for } i \neq j.$$

Using this representation for the transition probability and expanding the likelihood above, we have

$$L = \prod_{n=1}^N \left\{ \left(\lambda_{Z_{\tau_{n-1}}}^+ \right)^{M_{n-1}^+} e^{-\lambda_{Z_{\tau_{n-1}}}^+ (\tau_n - \tau_{n-1})} \left(\lambda_{Z_{\tau_{n-1}}}^- \right)^{M_{n-1}^-} e^{-\lambda_{Z_{\tau_{n-1}}}^- (\tau_n - \tau_{n-1})} \right. \\ \left. \times \Lambda_{Z_{\tau_{n-1}}} e^{-\Lambda_{Z_{\tau_{n-1}}} (\tau_n - \tau_{n-1})} [P]_{Z_{\tau_{n-1}} Z_{\tau_n}} \right\}.$$

Next, we re-arrange the terms by collecting like regimes and like transitions to obtain

$$L = \left(\prod_{i=1}^K (\lambda_i^+)^{\hat{M}_i^+} e^{-\lambda_i^+ \Delta \tau_i} \times (\lambda_i^-)^{\hat{M}_i^-} e^{-\lambda_i^- \Delta \tau_i} \right) \\ \times \left(\prod_{i,j=1}^K ([P]_{ij})^{n_{ij}} \right) \times \left(\prod_{i=1}^K \Lambda_i^{n_i} e^{-\Lambda_i \Delta \tau_i} \right).$$

Here, $\hat{M}_i^\pm$ denotes the number of market buy (sell) orders that occur while the chain is in regime i , $\Delta \tau_i$ denotes the total time the Markov chain spent in regime i , and n_{ij} denotes the number of times the chain switches from i to j . Since the times $\tau_1, \tau_2, \dots, \tau_N$ are by definition the times at which the Markov chain switches regime, we must have $n_{ii} = 0$ and so $[P]_{ii} = 0$.

Finally, we optimise the likelihood over the parameters $\lambda_i^\pm$, P and Λ_i . Each of these optimisations can be carried out independently, and follow along the lines outlined in the earlier subsection. Here, we simply record the final results for the MLE estimators of the model parameters:

$$\hat{\lambda}_i^\pm = \frac{\hat{M}_i^\pm}{\Delta \tau_i}, \quad \hat{\Lambda}_i = \frac{\sum_{j \neq i} n_{ij}}{\Delta \tau_i}, \quad \text{and} \quad [\hat{P}]_{ij} = \frac{n_{ij}}{\sum_{j \neq i} n_{ij}}, \quad (1)$$

from which we find that the MLE estimate of the off-diagonal elements of the generator matrix is

$$\hat{\lambda}_{ij} = \frac{n_{ij}}{\Delta\tau_i}. \quad (2)$$

The above results are similar to the simple Markov chain model, but now we also account for the regime specific arrival rate of market buy and sell orders.

2.2.5.1 Estimation on SOL-PERP

We apply Equation 1 to the full month of August 2024. The sufficient statistics are the total time $\Delta\tau_i$ the chain spends in each regime, the number of switches n_{ij} from i to j , and the counts $\hat{M}_i^\pm$ of market buy and sell orders arriving while the chain sits in regime i .

```
regime_time <- epochs[, .(
  delta_tau = sum(dur),
  n_epochs = .N,
  mean_epoch = mean(dur)
), keyby = regime]

switch_counts <- table(from = epochs$regime, to = epochs$to_regime)

mo_counts <- dcast(
  mo_events[, .N, by = .(regime, side)],
  regime ~ side,
  value.var = "N",
  fill = 0
)

joint_mle <- merge(regime_time, mo_counts, by = "regime")

# lambda_i^{+/-} = M_i^{+/-} / delta_tau_i
joint_mle[, `:=`(
  lambda_buy = buy / delta_tau,
  lambda_sell = sell / delta_tau,
  time_share = delta_tau / sum(delta_tau),
  Lambda = rowSums(switch_counts) / delta_tau
)]

# Off-diagonal entries lambda_ij = n_ij / delta_tau_i; diagonal is
# -Lambda_i.
```

```

B_joint <- sweep(unclass(switch_counts), 1, regime_time$delta_tau, FUN = "/")
diag(B_joint) <- -rowSums(B_joint)
dimnames(B_joint) <- list(regime_labels, regime_labels)

# Switching probabilities [P]_ij = n_ij / sum_j n_ij, with zero
# diagonal.
P_joint <- sweep(unclass(switch_counts), 1, rowSums(switch_counts), FUN = "/")

fnum <- function(x, d = 2) formatC(x, format = "f", digits = d, big.mark = "")

```

Two pieces of data handling are worth recording. First, as the derivation requires, the final epoch is dropped so that no censored observation enters the likelihood. Second, 5 epochs lasting longer than 60s, accounting for 722s in total, are feed outages rather than genuinely long-lived regimes; they are discarded along with the switch that ends them, since otherwise they would inflate $\Delta\tau_i$ and bias every rate downwards.

```

knitr::kable(
  joint_mle[, .(
    Regime = regime,
    `\$\\Delta\\tau_i$ (s)` = round(delta_tau),
    `Time share` = round(time_share, 4),
    Epochs = n_epochs,
    `Mean epoch (s)` = round(mean_epoch, 4),
    `\$\\hat{\\Lambda}_i$ (s$^{-1}$)` = round(Lambda, 3)
  )],
  escape = FALSE,
  format.args = list(big.mark = ",")
)

```

Table 6: Time spent in each regime and the switching rate out of it.

Regime	$\Delta\tau_i$ (s)	Time share	Epochs	Mean epoch (s)	$\hat{\Lambda}_i$ (s ⁻¹)
Very Negative	594,710	0.2221	4,480,455	0.1327	7.534
Negative	484,539	0.1810	6,211,467	0.0780	12.819
Neutral	503,754	0.1881	6,926,177	0.0727	13.749
Positive	491,834	0.1837	6,173,103	0.0797	12.551
Very Positive	602,841	0.2251	4,467,480	0.1349	7.411

```
knitr::kable(
  joint_mle[, .(
    Regime = regime,
    `$$\hat{M}_i^{+}$` = buy,
    `$$\hat{M}_i^{-}$` = sell,
    `$$\hat{\lambda}_i^{+}$` = round(lambda_buy, 3),
    `$$\hat{\lambda}_i^{-}$` = round(lambda_sell, 3),
    `$$\hat{\lambda}_i^{+}/\hat{\lambda}_i^{-}$` =
      round(lambda_buy / lambda_sell, 3)
  )],
  escape = FALSE,
  format.args = list(big.mark = ",")
)
```

Table 7: Market order counts and estimated regime-specific arrival rates.

Regime	$\hat{M}_i^{+}$	$\hat{M}_i^{-}$	$\hat{\lambda}_i^{+}$	$\hat{\lambda}_i^{-}$	$\hat{\lambda}_i^{+}/\hat{\lambda}_i^{-}$
Very Negative	636,881	1,620,737	1.071	2.725	0.393
Negative	518,412	658,324	1.070	1.359	0.787
Neutral	542,745	565,720	1.077	1.123	0.959
Positive	633,568	539,274	1.288	1.096	1.175
Very Positive	1,517,498	687,864	2.517	1.141	2.206

```
knitr::kable(B_joint, digits = 3)
```

Table 8: Estimated generator matrix $\hat{B}$ from the joint model, in units of s^{-1} .

	Very Negative	Negative	Neutral	Positive	Very Positive
Very Negative	-7.534	3.783	2.018	1.054	0.679
Negative	4.735	-12.819	4.690	2.108	1.286
Neutral	2.325	4.581	-13.749	4.531	2.313
Positive	1.246	2.076	4.603	-12.551	4.627
Very Positive	0.666	1.051	1.974	3.720	-7.411

```
rates_long <- melt(
  joint_mle[
    ,
```

```

      .(regime, `Market Buys` = lambda_buy, `Market Sells` = lambda_sell)
    ],
    id.vars = "regime",
    variable.name = "side",
    value.name = "rate"
  )

p_rates <- ggplot(
  rates_long, aes(x = factor(regime, levels = regime_labels),
    y = rate, fill = side)
) +
  geom_col(position = "dodge", alpha = 0.85) +
  scale_fill_manual(
    values = c("Market Buys" = "#1f77b4", "Market Sells" = "#ff7f0e")
  ) +
  labs(
    title = "Regime-Specific Market Order Arrival Rates",
    x = "Regime",
    y = "Arrival rate (market orders per second)",
    fill = "Side"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(size = 12, face = "bold"),
    axis.text.x = element_text(angle = 45, hjust = 1),
    panel.grid.major = element_line(color = "gray90"),
    panel.grid.minor = element_line(color = "gray95")
  )

print(p_rates)

```

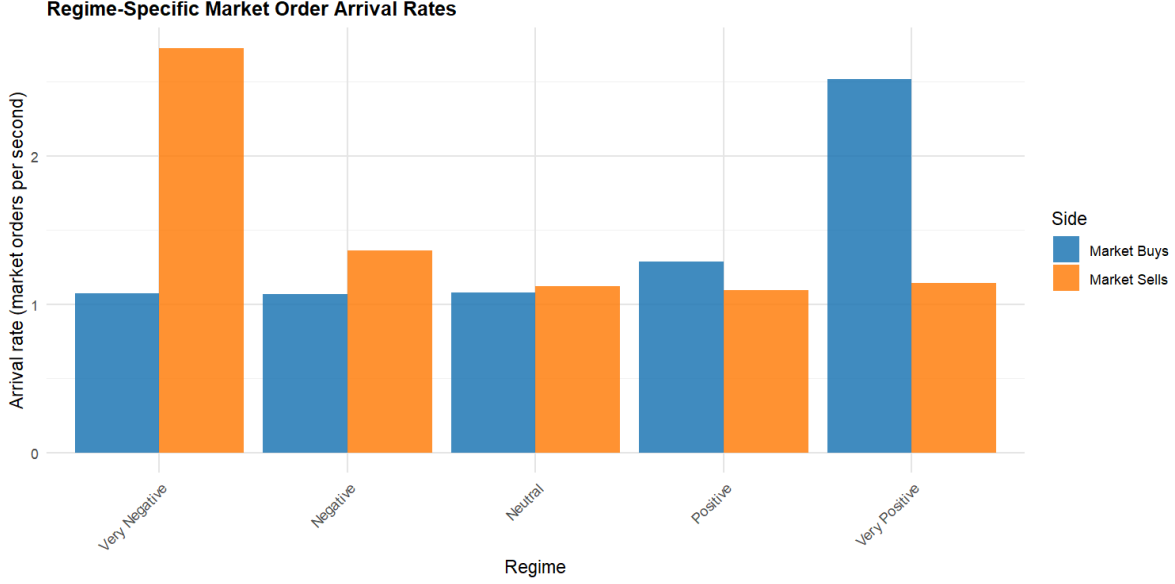


Figure 2: Estimated market order arrival rates by order imbalance regime. Bid-heavy regimes draw market buys, ask-heavy regimes draw market sells.

The arrival rates in Table 7 and Figure 2 are the main result. Unconditionally the month is close to balanced: 1.44 market buys and 1.52 market sells per second. Conditioning on the regime changes that picture entirely. In the most ask-heavy regime the sell rate is 2.73 against a buy rate of 1.07, and in the most bid-heavy regime the buy rate is 2.52 against a sell rate of 1.14. The ratio $\hat{\lambda}_i^+/\hat{\lambda}_i^-$ rises monotonically from 0.39 to 2.21 across the five regimes, so order imbalance is indeed a good predictor of the direction of order flow.

The shape of that dependence matters as much as its strength. The buy rate is essentially flat across the three left-most regimes and the sell rate is essentially flat across the three right-most ones; nearly all of the signal comes from the two extreme regimes, where the rate of the *aggressive* side roughly doubles rather than the passive side thinning out. In other words the imbalance is informative mainly when it is severe, which is precisely when a market maker quoting the thin side is most exposed.

The generator in Table 8 describes a much faster chain than the one in Section 2.2.3. Mean holding times $1/\hat{\Lambda}_i$ range from 0.073s to 0.135s, i.e. the chain switches roughly 10.6 times per second. The extreme regimes are about twice as persistent as the central ones, and the chain spends more time in them (22.2% and 22.5% of the sample) than in the neutral regime (18.8%): at the touch, the book is more often lopsided than balanced. Unlike a single equity name on a single day, $\hat{B}$ here is close to symmetric under the reflection $i \mapsto K + 1 - i$, so over a month there is no systematic pull towards a bid- or ask-heavy regime. Nor are switches confined to neighbouring regimes: out of the most ask-heavy regime only 50.2% of switches go to the adjacent one, while 23.0% jump straight across to a bid-heavy regime.

Comparing $\hat{B}$ with the estimate in Section 2.2.3, the overall rates differ by roughly two orders of magnitude: holding times there are 3.3s to 9.0s against fractions of a second here. That gap is an artefact of the smoothing, not a property of the market. The earlier estimate is built from a 10s trailing average of the 100ms-binned imbalance, and a moving average of that length cannot jump between regimes, which is why the resulting transition matrix is almost tridiagonal and so persistent. It also shows the cost of the embedding step: the matrix logarithm returns 8 negative off-diagonal entries, the most negative being -0.0031, so that $\hat{B}$ is not a valid generator, whereas Equation 2 delivers a valid one by construction. The event-time estimate is the one to use for a controller reacting at the millisecond scale; the smoothed estimate describes a slower, filtered version of the same signal.

2.2.6 Modelling price jumps

Up to this point, we have been concerned with understanding how order imbalance influences the rate of arrival of MOs. An important and interesting related question is to determine the distribution of price changes conditional on the arrival of buy and sell MOs and the order imbalance regime prior to the arrival of that MO. To answer this question, we record the time of each MO and compute the midprice change 1s afterwards, conditional on the order imbalance regime when split into 5 equal bins with knots at $\{-1, -\frac{3}{5}, -\frac{1}{5}, +\frac{1}{5}, +\frac{3}{5}, +1\}$, as in the previous section. The corresponding states and imbalance ρ is given by

$$Z = \begin{cases} -2, & \rho \in [-1, -\frac{3}{5}], & \text{sell-heavy,} \\ -1, & \rho \in [-\frac{3}{5}, -\frac{1}{5}), & \text{sell-bias,} \\ 0, & \rho \in [-\frac{1}{5}, +\frac{1}{5}), & \text{neutral,} \\ +1, & \rho \in [+ \frac{1}{5}, +\frac{3}{5}), & \text{buy-bias,} \\ +2, & \rho \in [+ \frac{3}{5}, +1], & \text{buy-heavy.} \end{cases}$$

Note that Z is a relabelling of the regimes used in the previous sections, $Z = i - 3$ for regime $i = 1, \dots, 5$, so that $Z = -2$ is the ask-heavy regime and $Z = +2$ the bid-heavy one.

The state Z and the reference midprice are taken from the most recent book update *strictly before* the MO's millisecond: an update stamped in the same millisecond as the trade may already reflect the trade itself, which would let the MO leak into the state we are conditioning on. The post-MO midprice is the most recent update at or before $t + 1000\text{ms}$. Everything is measured in ticks of \$0.001.

```
TICK <- 0.001

# One row per millisecond of book updates, for the rolling joins
# below.
book_mid <- bookTicker[
  transaction_time != shift(transaction_time, type = "lead", fill = -1),
```

```

    .(t = as.numeric(transaction_time), midprice, imbalance)
  ]
  setkey(book_mid, t)

mo_jumps <- data.table(
  t = as.numeric(trades$time),
  price = trades$price,
  side = fifelse(trades$is_buyer_maker, "sell", "buy")
)
setorder(mo_jumps, t)

# Roll each timestamp back to the most recent book update at or before
# it.
i_pre <- book_mid[.(mo_jumps$t - 1), roll = TRUE, which = TRUE]
i_before <- book_mid[.(mo_jumps$t - 1000), roll = TRUE, which = TRUE]
i_at <- book_mid[.(mo_jumps$t), roll = TRUE, which = TRUE]
i_100ms <- book_mid[.(mo_jumps$t + 100), roll = TRUE, which = TRUE]
i_1s <- book_mid[.(mo_jumps$t + 1000), roll = TRUE, which = TRUE]

mo_jumps[, `:=`(
  rho = book_mid$imbalance[i_pre],
  mid_pre = book_mid$midprice[i_pre],
  mid_before = book_mid$midprice[i_before],
  mid_at = book_mid$midprice[i_at],
  mid_100ms = book_mid$midprice[i_100ms],
  mid_1s = book_mid$midprice[i_1s]
)]
mo_jumps <- mo_jumps[complete.cases(mo_jumps)]

# Regime prior to the MO, on the knots of the previous sections: Z =
# -2, ..., +2.
mo_jumps[, Z := as.integer(
  cut(rho, breaks = quantile_bounds, include.lowest = TRUE)
) - 3L]

# Midprice change over the second following the MO, in ticks.
mo_jumps[, dS := (mid_1s - mid_pre) / TICK]

# Unconditional benchmark: the same 1s change sampled on a regular 1s
# grid.
grid_t <- seq(min(book_mid$t), max(book_mid$t) - 1000, by = 1000)
grid_dS <- (

```

```

    book_mid$midprice[book_mid[(grid_t + 1000), roll = TRUE, which = TRUE]] -
    book_mid$midprice[book_mid[(grid_t), roll = TRUE, which = TRUE]]
) / TICK
grid_dS <- grid_dS[!is.na(grid_dS)]

median_spread <- 2 * median(bookTicker$half_spread) / TICK

rm(book_mid, i_pre, i_before, i_at, i_100ms, i_1s)
invisible(gc())

```

SOL-PERP is not a \$33 stock quoted in cents, and the units matter for how the table is built. The tick is \$0.001 and the median quoted spread is 1 tick, but the midprice is far more volatile than that over a one-second horizon: sampled on a regular one-second grid, the central 90% of midprice changes spans -37 to 36 ticks. A row per tick would run to hundreds of rows, so we bucket ΔS instead, symmetrically about zero.

```

jump_levels <- c(
  "$\\Delta S \\le -20$", "$-20 < \\Delta S \\le -5$", "$-5 < \\Delta S < 0$",
  "$\\Delta S = 0$",
  "$0 < \\Delta S < 5$", "$5 \\le \\Delta S < 20$", "$\\Delta S \\ge 20$"
)

mo_jumps[, dS_bucket := factor(fcase(
  dS <= -20, jump_levels[1],
  dS <= -5, jump_levels[2],
  dS < 0, jump_levels[3],
  dS == 0, jump_levels[4],
  dS < 5, jump_levels[5],
  dS < 20, jump_levels[6],
  default = jump_levels[7]
), levels = jump_levels)]

# One panel per MO side: the conditional distribution of dS down the
# rows, with the arrival rate of that side from the joint model on
# top.
jump_panel <- function(mo_side, lambda, lambda_label) {
  d <- mo_jumps[side == mo_side]
  p <- prop.table(table(d$dS_bucket, d$Z), margin = 2)
  cells <- formatC(as.matrix(p), format = "f", digits = 3)
  cells[as.matrix(p) == 0] <- "--"
  out <- rbind(formatC(lambda, format = "f", digits = 3), cells)
  rownames(out) <- c(lambda_label, jump_levels)
}

```

```
colnames(out) <- paste0("$Z=", c("-2", "-1", "0", "+1", "+2"), "$")
out
}
```

```
knitr::kable(jump_panel(
  "buy", joint_mle$lambda_buy, "$\\hat{\\lambda}^{+}$"
),
  escape = FALSE
)
```

Table 9: Distribution of the midprice change 1s after a market **buy**, in ticks, conditional on the order imbalance regime. Columns sum to one; $\hat{\lambda}^+$ is the arrival rate from Table 7.

	$Z = -2$	$Z = -1$	$Z = 0$	$Z = +1$	$Z = +2$
$\hat{\lambda}^+$	1.071	1.070	1.077	1.288	2.517
$\Delta S \leq -20$	0.147	0.125	0.116	0.101	0.079
$-20 < \Delta S \leq -5$	0.076	0.069	0.062	0.057	0.046
$-5 < \Delta S < 0$	0.031	0.031	0.028	0.025	0.021
$\Delta S = 0$	0.088	0.069	0.056	0.048	0.027
$0 < \Delta S < 5$	0.051	0.058	0.057	0.056	0.047
$5 \leq \Delta S < 20$	0.135	0.161	0.178	0.199	0.219
$\Delta S \geq 20$	0.471	0.488	0.503	0.513	0.561

```
knitr::kable(jump_panel(
  "sell", joint_mle$lambda_sell, "$\\hat{\\lambda}^{-}$"
),
  escape = FALSE
)
```

Table 10: Distribution of the midprice change 1s after a market **sell**, in ticks, conditional on the order imbalance regime. Columns sum to one; $\hat{\lambda}^-$ is the arrival rate from Table 7.

	$Z = -2$	$Z = -1$	$Z = 0$	$Z = +1$	$Z = +2$
$\hat{\lambda}^-$	2.725	1.359	1.123	1.096	1.141
$\Delta S \leq -20$	0.564	0.511	0.496	0.484	0.461
$-20 < \Delta S \leq -5$	0.211	0.194	0.177	0.160	0.131
$-5 < \Delta S < 0$	0.049	0.056	0.058	0.058	0.051
$\Delta S = 0$	0.025	0.048	0.060	0.075	0.100
$0 < \Delta S < 5$	0.021	0.027	0.028	0.030	0.032
$5 \leq \Delta S < 20$	0.048	0.059	0.065	0.067	0.076

Table 10: Distribution of the midprice change 1s after a market **sell**, in ticks, conditional on the order imbalance regime. Columns sum to one; $\hat{\lambda}^-$ is the arrival rate from Table 7.

	$Z = -2$	$Z = -1$	$Z = 0$	$Z = +1$	$Z = +2$
$\Delta S \geq 20$	0.083	0.105	0.116	0.125	0.150

The first thing to note is what conditioning on an MO does to the atom at zero. On the one-second grid the midprice is unchanged 45.4% of the time; one second after a market order it is unchanged in only 5.0% of cases. Market orders arrive when the price is moving, and they are almost never followed by a flat second.

```
jump_summary <- mo_jumps[, .(
  n = .N,
  median_dS = median(dS),
  p_up = round(mean(dS > 0), 3),
  p_flat = round(mean(dS == 0), 3),
  p_down = round(mean(dS < 0), 3)
), keyby = .(side, Z)]

knitr::kable(
  jump_summary,
  escape = FALSE,
  col.names =
    c("Side", "$Z$", "MOs", "Median  $\Delta S$ ",
      "$P(\Delta S > 0)$", "$P(\Delta S = 0)$", "$P(\Delta S < 0)$"
    ),
  format.args = list(big.mark = ",")
)
```

Table 11: Median midprice change 1s after a market order and the sign probabilities, by side and regime.

Side	Z	MOs	Median ΔS	$P(\Delta S > 0)$	$P(\Delta S = 0)$	$P(\Delta S < 0)$
buy	-2	558,665	17.0	0.658	0.088	0.254
buy	-1	497,239	19.0	0.707	0.069	0.225
buy	0	570,086	20.0	0.738	0.056	0.206
buy	1	657,137	21.0	0.769	0.048	0.183
buy	2	1,567,022	24.0	0.827	0.027	0.146
sell	-2	1,661,437	-24.0	0.152	0.025	0.824
sell	-1	695,645	-21.0	0.190	0.048	0.762

Table 11: Median midprice change 1s after a market order and the sign probabilities, by side and regime.

Side	Z	MOs	Median ΔS	$P(\Delta S > 0)$	$P(\Delta S = 0)$	$P(\Delta S < 0)$
sell	0	597,872	-20.0	0.209	0.060	0.731
sell	1	519,723	-18.5	0.223	0.075	0.703
sell	2	598,393	-16.0	0.258	0.100	0.642

```
p_up_uncond <- mean(grid_dS > 0) / mean(grid_dS != 0)

p_jump <- ggplot(jump_summary, aes(x = Z, y = p_up, color = side)) +
  geom_hline(
    yintercept = p_up_uncond, linetype = "dashed", color = "gray40",
    linewidth = 0.4
  ) +
  geom_line(linewidth = 0.8) +
  geom_point(size = 2.5) +
  scale_color_manual(
    values = c(buy = "#1f77b4", sell = "#ff7f0e"),
    labels = c(buy = "Market Buys", sell = "Market Sells"),
    name = "Side"
  ) +
  scale_x_continuous(breaks = -2:2) +
  scale_y_continuous(limits = c(0, 1)) +
  labs(
    title = "Probability of an Up Move 1s After a Market Order",
    x = "Order imbalance regime Z",
    y = expression(P(Delta * S > 0))
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(size = 12, face = "bold"),
    panel.grid.minor = element_blank()
  )

print(p_jump)
```

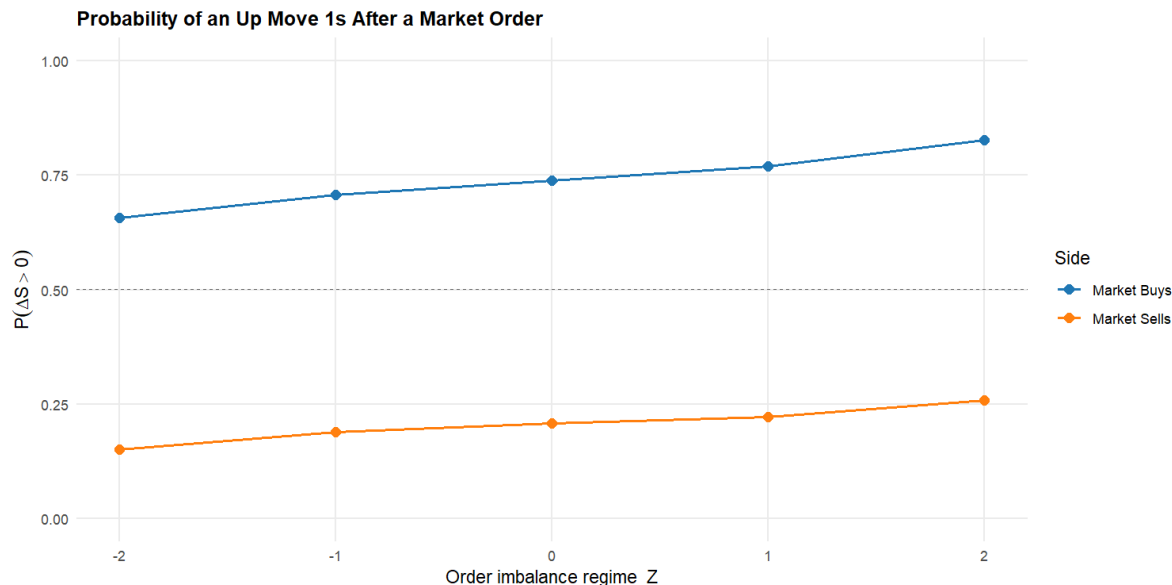


Figure 3: Probability that the midprice is higher 1s after a market order, by side and order imbalance regime. The dashed line is the unconditional probability of an up move, given the price moved at all.

The regime dependence is the point of the exercise, and Figure 3 shows it cleanly. Holding the side of the MO fixed, a more bid-heavy book shifts the whole distribution of ΔS upwards. After a market buy, the probability of an up move rises from 0.66 in the ask-heavy regime to 0.83 in the bid-heavy one, and the median change rises from 17.0 to 24.0 ticks. After a market sell, the probability of an up move rises over the same range from 0.15 to 0.26, and the median change from -24.0 to -16.0 ticks — a sell into a bid-heavy book does much less damage than the same sell into an already ask-heavy one. This runs in the same direction as the arrival rates of Section 2.2.5.1: imbalance predicts both who trades next and what the price does afterwards.

The medians and the sign probabilities are monotone in Z , but the conditional *means* are not: they are essentially flat across regimes, 32.4 ticks at $Z = -2$ against 31.8 ticks at $Z = +2$ for buys. The extreme regimes coincide with volatile bursts, so their tails are fat in both directions and dominate the average. The centre of the distribution is where the imbalance signal lives.

```
jump_horizons <- mo_jumps[, .(
  `1s before` = median((mid_pre - mid_before) / TICK),
  `At arrival` = median((mid_at - mid_pre) / TICK),
  `+100ms` = median((mid_100ms - mid_pre) / TICK),
  `+1s` = median((mid_1s - mid_pre) / TICK),
  `Print vs. mid` = median((price - mid_pre) / TICK)
), keyby = .(Side = side)]
```

```
knitr::kable(jump_horizons, digits = 1)
```

Table 12: Median midprice move around a market order, in ticks, measured from the last book update before it. The last column is where the trade itself printed relative to that midprice.

Side	1s before	At arrival	+100ms	+1s	Print vs. mid
buy	3.5	4	19	22.0	3.0
sell	-5.0	-4	-19	-21.5	-3.5

Three caveats belong with these numbers, all visible in Table 12. First, trades print a long way through the quoted book: a buy goes off a median 3.0 ticks, and a mean of 13.3 ticks, above the prevailing midprice in a market whose median spread is 1 tick. The quote feed lags the trade feed, so part of what follows a print is simply the book catching up to a price the trade has already revealed. Second, the midprice had already moved in the aggressor’s direction during the second *before* the trade, by a median 3.5 ticks for buys, so MOs are arriving mid-move rather than starting one. Third, most of the one-second move is over almost immediately: 85% of the mean move after a buy is realised within 100ms.

Taken together, ΔS measured this way is the price shift *associated with* the arrival of an MO — impact, the quote feed catching up, and the continuation of a move already underway — and not a forecast that could be traded from the moment of the print. What the lag does not contaminate is the comparison across regimes at a fixed side, since it applies equally in every column: that comparison is the content of Table 9 and Table 10.

3 Daily features

4 Optimal liquidation

4.1 Optimisation problem

4.1.1 Simulations